

Transfer Learning and Fine-Tuning

Learning Notes

1. Why Transfer Learning? A problem.

Say you have a case where you need a model to classify whether an x-ray or any other medical image shows signs of a disease or condition. Say this condition is rare, or for some reason, the amount of labelled images to train a model on is limited, such as 2000 images. With only 2000 images, it is impossible to train a deep neural network from scratch and have it be accurate enough to be considered useful. Most likely, you would require a large number of labelled images for training, in the hundreds of thousands or even millions. So what do you do?

1.1 The concept of transfer learning and fine-tuning

Transfer learning and fine-tuning is the practice of having a pretrained model and taking advantage of its pre-trained weights by performing modifications such as replacing the classification head and training that, or calibrating it (or fine-tuning it) to perform the new task you amend it for. So instead of training a model from the beginning, you begin with parameters already set, essentially providing you with a launchpad that allows you to get a functioning model on the new task that might otherwise not have been possible to do so.

It is important to keep in mind that the bigger the difference between what the model was trained to perform and what it is being fine tuned to perform, the more data you most probably need for fine-tuning to get a decent model, the more time, and the riskier it is (as the modifications to the pretrained model become more aggressive).

1.2 An Analogy. Language

Think of yourself as someone learning a new language when you already know one. Say you know how to speak and write Italian fluently. Now say you want to learn Spanish. Chances are you will find it considerably easy. Why? Because both languages use essentially the same alphabet (maybe a few added and removed accents/letters), both languages use the same syntax and both languages share a considerable amount of vocabulary (or slight alterations to the words). They are both languages with Latin roots. There is a considerable amount of overlap between the two, so you do not need to learn things from scratch. In that case 'fine-tuning' yourself to learn Spanish when you know Italian will not require too much change or time or information, because a good chunk of what you already learned (your internal parameters) you will mostly reuse as is or require very minimal modifications.

If you were to learn english, you will still mostly use the same alphabet and syntax overall. The vocabulary would be considerably different, meaning you would have to 'forget' some Italian words (this

assuming you can only keep 1 language in your head) and learn the new words for English. This would require a more 'aggressive' fine-tuning than Spanish.

If you had to learn Chinese, then most of what you already know regarding the language you know you'd need to forget. New alphabet, new syntax, new vocabulary. Even how to speak it requires you to do a considerable amount of change. This might require even more 'aggressive' fine-tuning, or you might as well forget everything you know and start from scratch, as previous knowledge might prevent you from grasping new knowledge cleanly.

2. How neural network learn

Before moving on to different transfer learning and fine-tuning techniques, it's best to understand more or less how neural networks work and what pattern emerges in their layers when trained. Transfer learning is possible because the layers in a network end up performing different 'tasks', specifically, early layers (closer to the input layer) tend to perform more general tasks (such as in the case of computer vision, recognizing edges) whilst later layers (closer to the output/classification layer) tend to perform more specific tasks (recognizing a cat from a dog). This behaviour is emergent rather than designed.

2.1 General to specific hierarchy

A deep neural network is composed of multiple layers, where one layer takes the output of the previous layer as input, performs some transformation on that input and forwards its output to the next layer. Even though in most cases the architecture is fairly uniform (different layers are very similar in shape and often perform the same arithmetics and such), what each layer learns to 'predict' is not uniform throughout.

Yosinski et al. demonstrated this in 2014 by literally measuring how transferable the features at each layer were. Zeiler and Fergus (2014) visualised the same hierarchy directly, using a deconvolution technique to show what kinds of patterns each layer responded to. Both of these findings have been observed across many architectures of neural networks as well as different datasets. As a result, although not always the case, this emergent behaviour has become somewhat a defacto assumption.

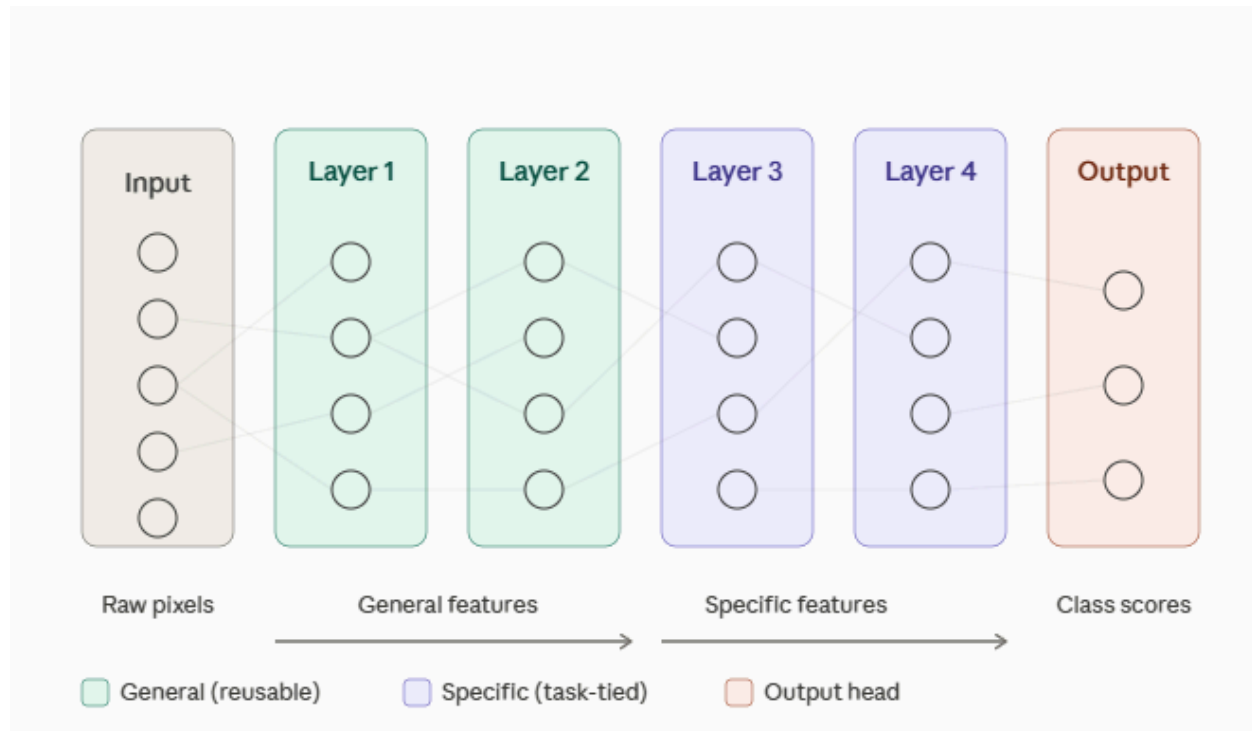


Figure 1: Layer distribution.

2.2 Vocabulary

Some vocabulary that will come in useful when understanding the notes and when doing any further research.

- **Transfer learning:** The general practice of using knowledge gained from one task to improve performance on another. It is an umbrella term that includes everything else in this list.
- **Pretraining:** The original training of a network on a previous dataset. ImageNet pretraining is the most common example in computer vision.
- **Backbone:** The pretrained part of the model, minus the final classification layer (NOTE: this is almost always replaced when transfer learning and fine-tuning).
- **Head:** The final classification layer, typically a single linear layer that maps the backbone's feature vector to your target classes. The head is usually replaced when adapting to a new task because the source and target classes differ.
- **Fine-tuning:** Continuing to train some or all parameters of a pretrained network on a new target dataset.
- **Domain adaptation:** A related but distinct subfield concerned specifically with handling differences between source and target distributions, often without labeled target data. Touched on briefly in the section on domain shift.

3. The techniques

There are 4 techniques covered here. Each will be explained, and when it might be best to use it. Note that in every technique, the Head of the pretrained model must always be replaced, whilst the input layer must remain the same. I highlight it here just to remind you.

3.1 Frozen Feature Extraction

This is the simplest form of transfer learning. Here, you take a pretrained model, freeze all parameters in all its layers (so you freeze the backbone), remove the final classification layer, and attach a new randomly initialized (or however it is best to initialize) head. Then you only train the new head. The frozen backbone acts as a feature extractor. In essence, the network is used to extract the vectors for the features of the new training/fine-tuning data, and the new head is trained to predict an outcome out of those. This is because the backbone is frozen, so the input vectors will always be the same.

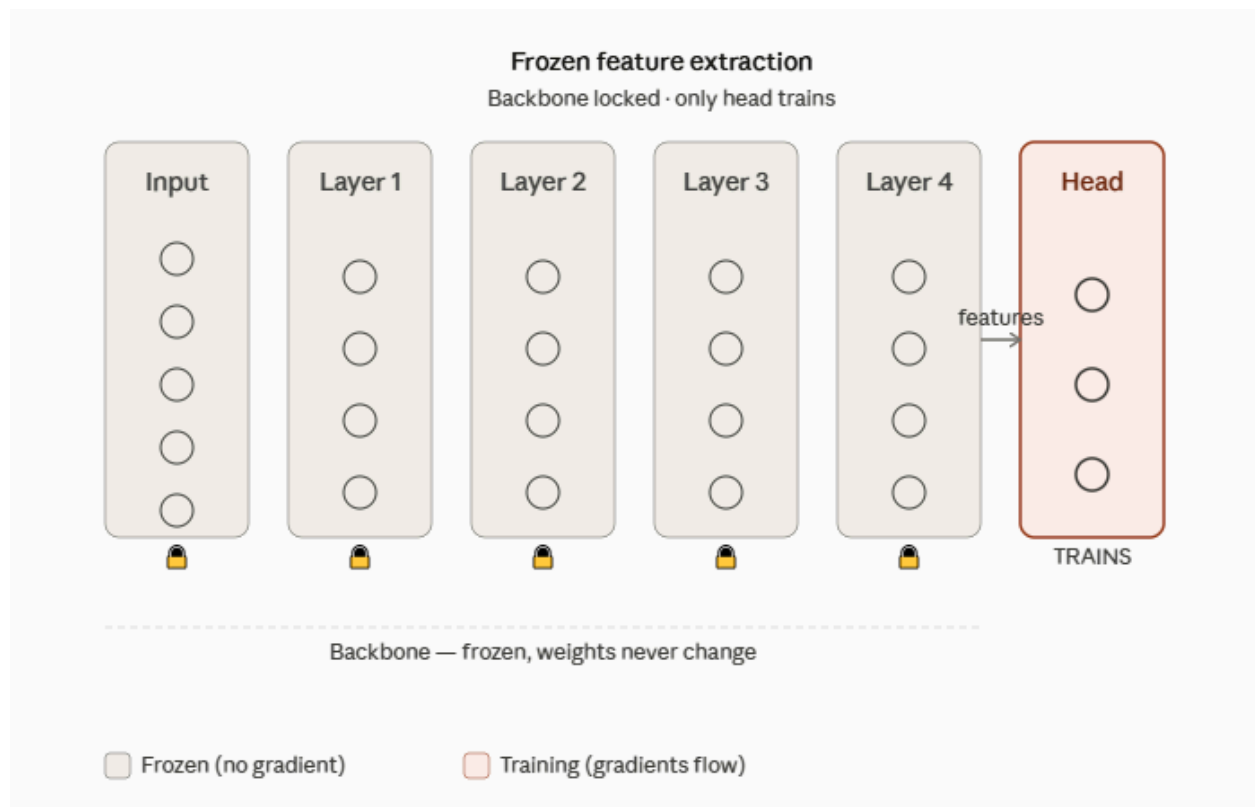


Figure 2: Frozen Feature Extraction technique.

For example, ResNet50, a CNN trained on ImageNet, has a representation space of 2048 dimensions. The new head simply has to learn how to draw boundaries for classification in that space. In simpler terms, you do not change how the network sees the world but rather you simply change how it labels what it sees.

This technique is most applicable when the new dataset you want to fine-tune the model on is small, and the domain of the new dataset is similar to the dataset the network was pretrained on. For example, the pretrained model was trained to classify cats from dogs, and you want to fine-tune it to classify Labradors from Pitbulls.

3.1.2 A note.

This technique is often used as the baseline when it comes to fine-tuning/transfer learning with other techniques. It is the simplest, fastest, and least invasive fine-tuning technique, as the pretrained network is (apart from the Head) unchanged. So it is often best to start with this technique before moving on to potentially more invasive fine-tuning.

3.2 End-to-End Fine-Tuning

In end-to-end fine-tuning, a new head is initialized as in the previous technique and replaces the previous head on the backbone. However, the backbone does not stay frozen. Every weight/parameter inside the backbone is trained alongside the new Head. The training/fine-tuning often uses a much lower learning rate than would be used if the model was trained from scratch. The idea here is that the pretrained weights that are valuable end up not being changed much, if at all, whilst the ones that require the most change are gradually overwritten.

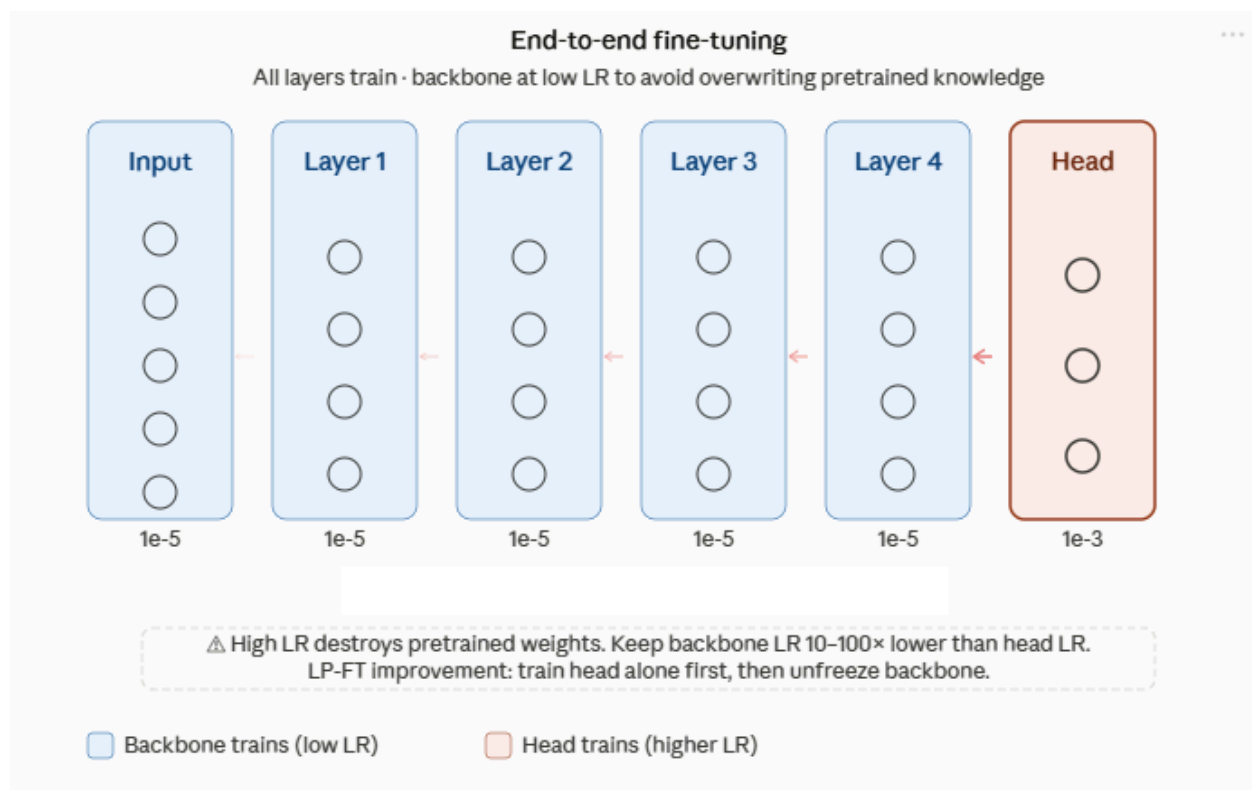


Figure 3: End-to-End Fine-Tuning

This is the most aggressive fine-tuning technique, and the one most prone to a condition called ‘Catastrophic Forgetting’. Catastrophic forgetting is the scenario where the backbone’s pretrained weights are changed to the point where there is no benefit taken from having them pretrained, and the pretrained model that has been modified heavily often performs worse than if the model was trained from scratch.

As mentioned, it is often best to use a learning rate that is 10x to 100x smaller than the one used when the pretrained model was trained from scratch. This is small enough that the pretrained weights are nudged towards the newly preferred value rather than overwritten so heavily.

In simple terms, end-to-end fine-tuning is training a model that, rather than having its weights be randomly initialized, has already somewhat stable weights, and you use a smaller learning rate.

This technique (and the subsequent LP-FT technique) is the most aggressive as mentioned earlier, and is best used when the domain difference between the pretrained model and what you want it to do is considerable and you have a considerable amount of data to train/fine-tune on.

3.2.1 In Distribution (ID) and Out of Distribution (OOD)

Two important terms worth mentioning now. In Distribution means that the test data comes from the same distribution as the training/fine-tuning data. For example, both test and train/fine-tuning data use the same lighting conditions, same image quality, same object to classify/detect, etc. Out of Distribution means the test data comes from (potentially) a related but different distribution. For example, different lighting, different image quality, different times of day, but the same object to classify/detect. In most cases, real-world deployed models will always be used during inference on OOD data relative to what they were trained/fine-tuned on.

3.2.2 ID/OOD tradeoff

A paper showed that the end-to-end fine-tuning technique described above produced certain behaviors in models when tested on ID vs OOD data. Kumar et al. showed that, compared to Frozen Feature Extraction (the first technique explained), models tuned with end-to-end fine-tuning performed better on ID data but worse on OOD data. On further observation, they noticed that the newly initialized head (often randomly initialized) during fine-tuning was sending chaotic gradients back to the backbone layers, causing the pre-trained weights to change drastically, even with a lowered learning rate. Which brings us to their solution.

3.2.3 Linear Probe then Fine Tune

What Kumar et al proposed was simple. First, keep the backbone frozen and train the newly initialized head until it is in a sensible space. Then unfreeze the backbone and train everything end-to-end. This way, the somewhat stable head will not send chaotic gradients into earlier layers during Back Propagation.

3.3 Gradual Unfreezing

Gradual unfreezing lies somewhere between Frozen Feature Extraction and End-to-End Fine-tuning. Rather than updating both the Head and the backbone all at once, as in End-to-End Fine-tuning, Gradual unfreezing starts with the backbone frozen and slowly unfreezes layer after layer, from the layer closest to the Head all the way (or until desired) to the earliest layer. What happens is you start with the backbone frozen and train the Head for a defined number of epochs. After those epochs have passed, unfreeze the layer before the head (or how many layers you wish), train those + the already unfrozen Head for another amount of epochs, then unfreeze the layer/s before the most recently unfrozen. This continues until every layer (or until the layer you want) is updated.

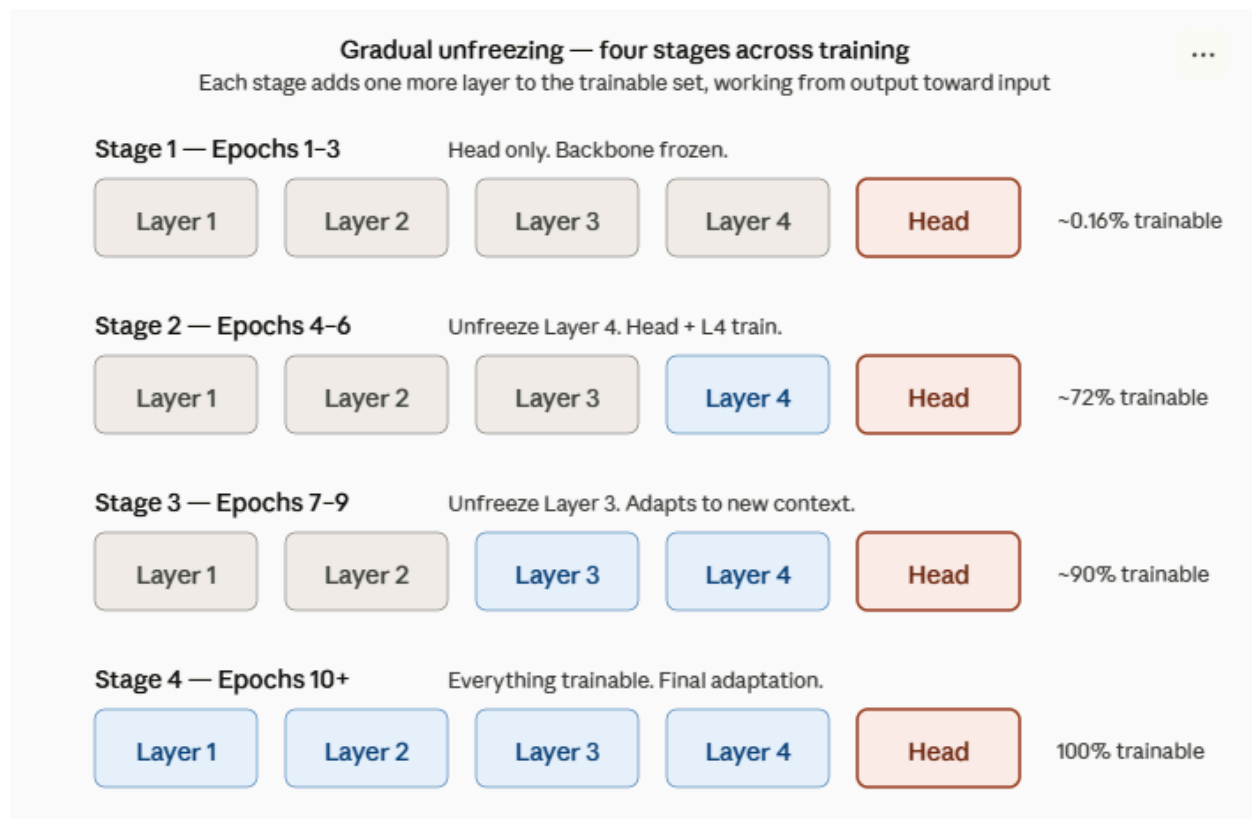


Figure 4: Gradual Unfreezing

This technique acts almost like LP-FT, but for every layer/batch of layers, meaning that newly unfrozen layers get gradients to update themselves from layers that have already somewhat stabilized.

This technique was introduced by Howard and Ruder in the 2018 ULMFit paper, which is one of the papers referenced in the project. The paper itself was originally about Natural Language Processing and used an LSTM rather than a CNN, but again, these techniques are, for the most part, architecture-agnostic.

When it comes to the number of epochs to choose before unfreezing the next layers to tune, there is no specific amount. The common amounts are between 1 and 5 epochs per stage or unfreezing subsequent layers when the loss plateaus.

This technique is most useful when you have a moderate amount of data and a moderate difference between the dataset/domain the model was pre-trained on and the new dataset/domain to fine-tune for.

3.4 Discriminative learning rates

Discriminative learning rate is a technique that is somewhat similar to Gradual Unfreezing and End-to-End Fine-Tuning and is, in fact, used alongside Gradual Unfreezing. The technique was proposed in the same paper as Gradual Unfreezing.

In this technique, rather than gradually unfreezing each layer as in Gradual Unfreezing, or training/fine-tuning the backbone + Head with a single learning rate, each section (or even layer) of the backbone uses a different learning rate, with earlier layers using a smaller one. This is done as discussed earlier; earlier layers tend to be general and do not require drastic changes, at least not as much as later layers, which become more specific the closer they are to the output layer/Head. As a result of this, this technique proposes using a more aggressive learning rate for later layers since they require the most change, and smaller ones for earlier layers.

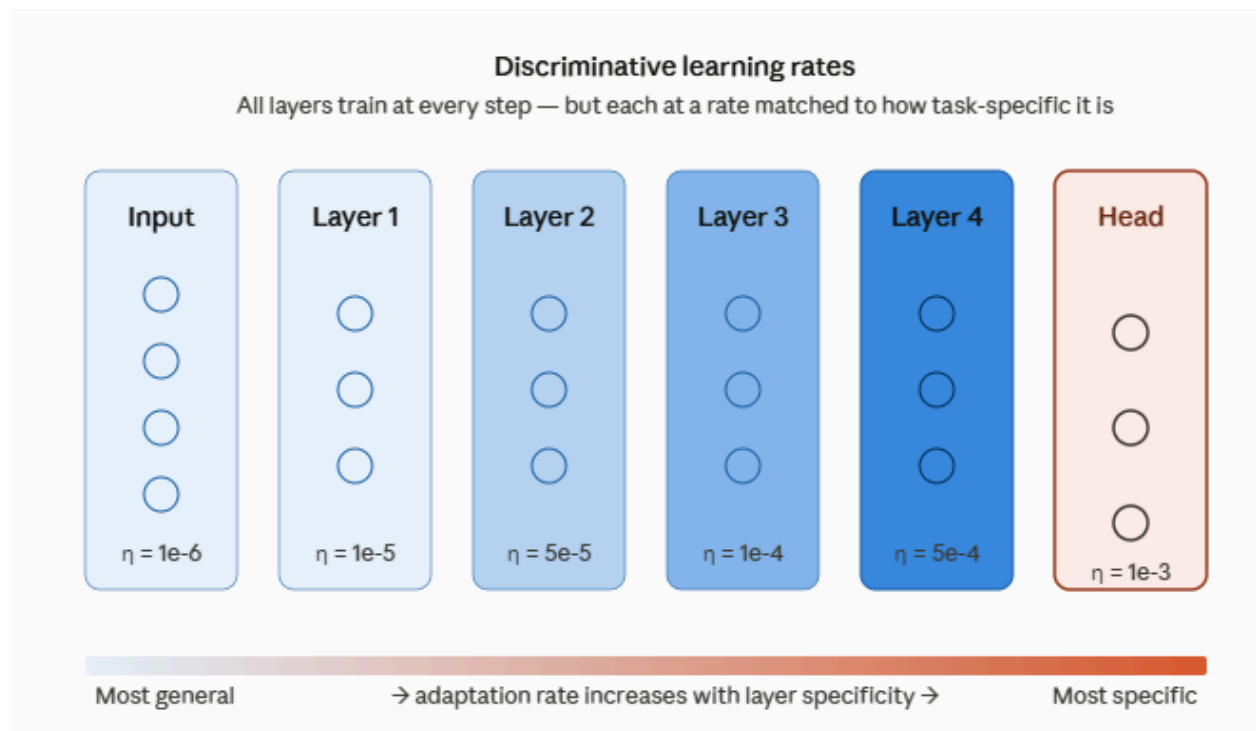


Figure 5: Discriminative Learning Rates

Howard and Ruder proposed the following formula to calculate the value of the learning rate to be used per layer:

$$\eta_l = \eta_L / \delta^{(L-l)}$$

where η_L is the learning rate of the final layer, L is the total number of layers, l indexes the current layer, and δ is a decay factor. Howard and Ruder used $\delta = 2.6$ for NLP. In computer vision, a larger decay factor of 10 is used between layer groups rather than per layer. But again, these are hyperparameters that, if possible, should be experimented with and evaluated.

To reiterate, Discriminative learning rates and gradual unfreezing are complementary rather than exclusive techniques. In ULMFiT, both are used together, where Gradual Unfreezing allows newly unfrozen layers to receive stable gradients from already stable layers, whilst discriminative learning rates make use of different learning rates based on the layer/group of layers.

Since this technique is complementary to Gradual Unfreezing, the best use of it is in the same scenario as Gradual Unfreezing.

3.5 Disclaimer

The techniques mentioned above are not exclusive from each other. LP-FT for example uses the concept of Frozen Feature Extraction to stabilize the Head before tuning end to end, whilst Gradual Unfreezing and Discriminative learning rates are themselves proposed as complementary techniques. Feel free to experiment and use any combination as you best see fit based on the circumstance/preference.

4. Domain Shift

In essence, Domain shift is one of the main reasons for transfer learning and fine-tuning. Domain shift (also known as Distribution Shift) refers to the mismatch between the data a model is trained on and the data it encounters during deployment. A computer vision model trained on ImageNet, which is fairly common, is trained to label everyday objects for the most part. Using this model to identify pneumonia in X-ray images would not result in very accurate (if at all) labeling/identification/classification. Or trying to use the model to identify objects that it has seen in high-quality images of ImageNet, from grainy pictures would have the model struggle. Those are domain shifts. Therefore, you would need to fine-tune or perform some sort of transfer learning.

There are multiple types of Domain Shifts, with the following being the most prominent.

4.1 Covariate Shift

The input distribution changes, but the relationship between the inputs and the labels stays the same. For example, as I mentioned, a model trained to identify cars that was trained on high-quality images, used to identify cars from grainy, dark images.

4.2 Label Shift

The class distribution changes. This is where a model might be trained to classify between cats and dogs, using a dataset with 50% dog images and 50% cat images. Then it is deployed and 90% of the time it gets images of cats.

4.3 Concept drift

The mapping between the input and the labels changes completely. For example, a model that was trained to detect spam emails in 2010 was used to detect spam emails in 2026. Spam emails in 2010 might have contained scams related to Nigerian princes needing money, whilst 2026 spam emails might contain 'quick crypto money grabs'. This is the hardest scenario to address through fine-tuning. It requires a considerable amount of data to fine-tune, and very aggressive fine-tuning is required. It might be worth simply training a network from scratch.

There is a specific field called 'Domain Adaptation' which proposes other tuning techniques, but it was beyond the scope of these notes. It is worth exploring, however.

5. Practical examples

Here, we will have a look at what it would look like from an applied numerical perspective if we fine-tune a ResNet18 (which we do in the simulator) model using each of the 4 techniques mentioned above.

5.1 Setup

We adapt a ResNet18 pretrained on ImageNet to the Oxford-IIIT Pet dataset, which contains 7400 images across 37 cat and dog breeds. ResNet18 has about 11.7 million parameters and produces a 512-dimensional feature vector before its final classification layer. We replace its 1000-class ImageNet head with a new 37-class head.

The new head is a single-layer mapping 512 inputs to 37 outputs:

$$(512 \times 37) + 37 = 18981 \text{ parameters}$$

This is $18981 / 11689512$, which equals about 0.16% of the total parameter count. This ratio will help highlight specific aspects of each technique's application.

5.2 Frozen Feature Extraction

Here, only the head is trainable. The 11+ million backbone parameters are frozen and never change.

Because the backbone never changes, every image always produces the same feature vector. This means we can run all 7400 images through the backbone once, cache the resulting tensor to disk, and train the head entirely on those cached vectors. Training the head, say, for 30 epochs on cached features takes seconds on a CPU. Running the full backbone for the same 30 epochs would take much, much longer.

5.3 Full End-to-End Fine-Tuning

Every parameter is updated. The backbone is updated alongside the head.

The learning rate choice is critical. If we were to use Stochastic Gradient Descent, each step update is:

$$w_{new} = w_{old} - \eta \cdot g$$

With typical pretrained weight magnitude 0.05 and gradient magnitude 1.0, at $\eta = 1e-3$, one step changes the weight by 2% of its magnitude. This will overwrite the preexisting weight in just a few hundred steps. At $\eta = 1e-5$, the same step moves the weight by only 0.02%. This will nudge the weight toward the correct value without overwriting it outright (preventing catastrophic forgetting). This is why fine-tuning uses learning rates 10x to 100x smaller than those used when training from scratch.

Small reminder: Even with a reduced learning rate, a randomly initialized head sends chaotic gradients into the backbone during early training. It is highly recommended to use LP-FT to first freeze the backbone and train the head until it stabilizes, then unfreeze and fine-tune end-to-end. This prevents the uninitialized head from disrupting pretrained weights before it has learned anything useful.

5.4 Gradual Unfreezing

The backbone starts out fully frozen. Layers/Layer groups are unfrozen one at a time, from closest to the head back toward the input, whilst the head and any already unfrozen layers continue to train at each stage.

For ResNet18 in this example, with this schedule:

Phase	Epochs	Trainable params	% of total
Head only	1–5	18,981	0.16%
+ Late backbone	6–15	~10,519,000	~90%
+ Early backbone	16–30	~11,689,512	100%

The logic is similar to LP-FT but applied layer by layer. Each newly unfrozen group receives gradients from layers that have already stabilized rather than from a randomly initialized head. Early layers, which are the most general and transferable, are left frozen the longest.

Again, there is no fixed rule for the number of epochs per stage. Anywhere from 1 to 5 is common or unfreeze the next group when the loss plateaus.

Reminder: Gradual unfreezing and discriminative learning rates are complementary. Unfreezing controls when each group starts training, and discriminative rates control how fast. Howard and Ruder proposed both in the same paper (ULMFiT, 2018) and intended them to be used together. In these study notes, they are separated to explain and display them as they are more easily.

5.5 Discriminative Learning Rates

Rather than one learning rate for the whole network, each layer/layer group gets its own. Earlier layers learn general features that transfer well and need little adjustment, whilst later layers are more task-specific and need more. So earlier groups get lower rates whilst later groups get larger ones.

If we were to group ResNet18's layers into 3 groups as follows:

Group	Layers	Learning rate
Early backbone	conv1, layer1, layer2	1e-5
Late backbone	layer3, layer4	1e-4
Head	fc	1e-3

Howard and Ruder proposed a formula for computing rates per layer

$$\eta_l = \eta_L / \delta^{(L - l)}$$

where η_L is the final layer rate, L is the total number of layers, l is the current layer index, and δ is a decay factor of 2.6 in the original NLP setting, typically 10 between layer groups in computer vision.

Under a unit gradient and a 0.05 reference weight, the early backbone weights move 0.02% per step, the late backbone 0.2%, and the head 2%. Over 100 epochs the early backbone barely shifts whilst the late backbone adapts substantially and the head converges.

6. Real-World Applications

The following are applications of Transfer Learning and Fine-Tuning in different fields.

6.1 Medical imaging

CheXNet, which was published by Rajpurkar et al. in 2017, used a 121-layer DenseNet pretrained on ImageNet and fine-tuned on the NIH ChestX-ray14 dataset (over 100,000 frontal chest X-rays) to detect 14 thoracic diseases. The system ended up reaching radiologist-level performance on pneumonia detection.

With over 100,000 labelled examples available the dataset was large enough to support full backbone adaptation which the authors found was necessary to achieve diagnostic accuracy.

6.2 Satellite imagery

The domain is considerably different from ImageNet as the images are of a top-down view, of different scales and often non-RGB spectra. The He et al. (2019) finding becomes relevant here. On large enough

satellite datasets used for dense prediction tasks like land-use segmentation or building detection, training from scratch can match or exceed ImageNet pretraining. Pretraining here helped mainly by accelerating early convergence rather than by providing useful features.

7. Paper references

The following papers are references throughout the notes or are relevant enough that further reading on them would be beneficial.

Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014) How transferable are features in deep neural networks? In Advances in Neural Information Processing Systems 27 (NeurIPS 2014), pp. 3320–3328. arXiv:1411.1792.

URL: <https://arxiv.org/abs/1411.1792>

This is the foundational empirical study of how features transition from general to task-specific across the layers of a deep network. The authors take an AlexNet-style CNN trained on ImageNet, split it at every possible depth and measure the accuracy cost of either freezing transferred layers or fine-tuning them on a related task. They identify two distinct issues that hurt transferability which are fragile co-adaptation between neighbouring layers, which causes optimisation problems when networks are split between layers that learned to work together, and specialisation of higher layers to the original task. They also show that fine-tuning transferred layers recovers and often exceeds original performance.

Howard, J., & Ruder, S. (2018) Universal Language Model Fine-tuning for Text Classification. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018), pp. 328–339. arXiv:1801.06146.

URL: <https://aclanthology.org/P18-1031/>

As highlighted earlier, this paper was originally about NLP, with ULMFiT being the source for two of the techniques covered in this topic (Gradual unfreezing and discriminative learning rates). The paper explains the combination of these two techniques where each layer is assigned its own learning rate via exponential decay from output toward input and layers are gradually unfrozen. The paper also introduces slanted triangular learning rates as a complementary scheduler. Howard and Ruder demonstrate that combining these techniques on an LSTM-based language model reduces error by 18–24% across six text classification benchmarks while requiring far less labelled data than training from scratch (which is beneficial as explained in the beginning).

Kornblith, S., Shlens, J., & Le, Q. V. (2019) Do Better ImageNet Models Transfer Better? In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2019), pp. 2661–2671. arXiv:1805.08974.

URL: <https://arxiv.org/abs/1805.08974>

This paper conducts an empirical study comparing 16 ImageNet-pretrained architectures across 12 downstream classification datasets under both frozen feature extraction and full fine-tuning. The authors found a strong correlation between ImageNet accuracy and downstream transfer accuracy (r

= 0.99 for frozen feature extraction and $r = 0.96$ for fine-tuning). They also document important caveats such as fine-grained tasks benefit less from transfer learning and certain ImageNet regularisation techniques such as label smoothing actually hurt frozen-feature transfer because they collapse class-discriminative information needed by downstream tasks.

He, K., Girshick, R., & Dollár, P. (2019) Rethinking ImageNet Pre-training. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2019), pp. 4918–4927. arXiv:1811.08883.

URL: <https://arxiv.org/abs/1811.08883>

This paper acts more as an adversary to the benefits of transfer learning and fine-tuning. It demonstrates that on COCO object detection and instance segmentation, models trained from random initialisation match the accuracy of ImageNet-pretrained counterparts when given sufficient training time and proper normalisation. The result holds even when using only 10% of the COCO training data, and across multiple model sizes and tasks. The authors conclusion is that ImageNet pretraining accelerates convergence early in training and helps under very-low-data regimes. However it does not provide regularisation benefits or improve final accuracy on dense prediction tasks given enough training.

Kumar, A., Raghunathan, A., Jones, R., Ma, T., & Liang, P. (2022) Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution. In International Conference on Learning Representations (ICLR 2022, Oral). arXiv:2202.10054.

URL: <https://arxiv.org/abs/2202.10054>

The authors here check across ten distribution-shift benchmarks and show that full fine-tuning achieves higher in-distribution accuracy than linear probing on frozen features but lower out-of-distribution accuracy, by 7% on average. They explain the mechanism both theoretically (proving the OOD bound for two-layer linear networks) and empirically that when the classification head is randomly initialised and you fine-tune everything immediately, the large early gradients flowing from the random head distort the well-learned feature extractor before the head has stabilised. They propose LP-FT, a two-stage strategy of first training the head on frozen features and then fine-tuning the entire network. This is a considerably important paper, highly recommend skimming through it at least.

Zeiler, M. D., & Fergus, R. (2014) Visualizing and Understanding Convolutional Networks. In Computer Vision – ECCV 2014, Lecture Notes in Computer Science vol. 8689, pp. 818–833. Springer. arXiv:1311.2901.

URL: <https://arxiv.org/abs/1311.2901>

This paper is more to gain a better understanding of CNNs. It introduces the deconvolution-based visualisation technique that produced the now-iconic images of what each layer of a CNN responds to: edges in early layers, textures in middle layers, parts in deeper layers, whole-class compositions at the top. The paper also presents ZFNet, an architectural refinement of AlexNet derived from insights gained through the visualisations, which improved on the state-of-the-art ILSVRC results.

Transfer Learning & Fine-Tuning Cheat Sheet

The Core Idea

Take a pretrained model, replace its head, and adapt it to your new task instead of training from scratch. Works because early layers learn general features (edges, textures) that transfer well, late layers learn task-specific features that often don't.

Key Vocabulary

Once again, just as a reminder

- **Backbone** - pretrained model minus the final classification layer
- **Head** - the final classification layer; always replaced when adapting to a new task
- **Pretraining** - original training on a large source dataset (e.g. ImageNet)
- **Fine-tuning** - continuing to train some/all parameters on your new dataset
- **Catastrophic forgetting** - when fine-tuning overwrites pretrained weights so heavily that the model performs worse than training from scratch

The 4 Techniques (Least to Most Invasive)

Technique	What you train	When to use
Frozen Feature Extraction	Head only; backbone frozen	Small dataset + similar domain. Best baseline.
Gradual Unfreezing	Head first, then unfreeze layers progressively from head to input	Moderate data + moderate domain shift
Discriminative Learning Rates	Everything, but each layer/group gets its own LR (smaller for earlier)	Used alongside Gradual Unfreezing
End-to-End Fine-Tuning	Every parameter, with a low LR (10x to 100x smaller than from-scratch)	Large dataset, especially if domain shift is significant

Critical Tips

- **LP-FT (Linear Probe then Fine-Tune)**: Always consider this. Freeze backbone, train head until stable, then unfreeze and fine-tune end-to-end. Prevents the randomly initialized head from sending chaotic gradients into pretrained layers (Kumar et al., 2022).
- **Learning rates for fine-tuning**: 10x to 100x smaller than from-scratch training. Typical: backbone at $1e-5$ to $1e-4$, head at $1e-3$.
- **Discriminative LR formula**: $\eta_L = \eta_{L'} / \delta^{(L - L')}$. $\delta \approx 2.6$ (NLP), $\delta \approx 10$ between groups (CV).

- **Gradual unfreezing + Discriminative LRs** are complementary, not alternatives. ULMFiT used both.

ID vs OOD Tradeoff (Kumar et al.)

- **In-Distribution:** test data matches training distribution
- **Out-of-Distribution:** test data is related but different (different lighting, camera, etc.) - what real deployments usually face
- **Full fine-tuning** - better ID, worse OOD
- **Frozen features** - worse ID, better OOD
- **LP-FT** - recovers most of both

Domain Shift (the reason fine-tuning exists)

Type	What changes	Fix
Covariate Shift	Input distribution (e.g. lighting, image quality)	Fine-tuning handles this well
Label Shift	Class proportions (e.g. 50/50 to 90/10)	Reweighting. Rarely needs retraining
Concept Drift	The actual input to label relationship	Hardest. Often better to retrain from scratch

Decision Framework for which technique to use.

- Start with Frozen Feature Extraction as a baseline - fastest, safest, most informative
- If accuracy is too low, then try LP-FT (frozen head training, then unfreeze)
- If domain shift is significant, then add Gradual Unfreezing + Discriminative LRs
- If you have lots of data and a very different domain, then consider full end-to-end fine-tuning, or even training from scratch (He et al., 2019)

Common Misconceptions

- **"Freezing disables a layer"** : No, it still runs forward, just doesn't update
- **"More compute = always fine-tune everything"** : No, often hurts OOD
- **"Pretraining always helps"** : No, sometimes only accelerates convergence (He et al., 2019) and sometimes from scratch is just better.
- **"More trainable params = better adaptation"** : No, must match dataset size or you overfit
- **"Gradual Unfreezing OR Discriminative LRs"** : No, use both